`PasswordHasher`, to protect data, even if you have no intention of leveraging Identity-based storage or UI. Another item that comes under the heading of cryptographic failures is failure to enforce TLS for data transfer, which we've already explored in the first section of the chapter.

13.3.1 Plain-text passwords

Rule number one in the web security handbook (if there is such a thing) is never store user passwords in plain text. People are creatures of habit. Many of them will use the same password on your site that they use for every other site they have an account with. In the event of a data breach, you could be giving crooks the keys to all other websites your users access. The website Have I Been Pwned (https://haveibeenpwned.com/) lists over 500 websites that have suffered data breaches, many of them familiar names and a significant number of them yielding up passwords in plain text.

So how should you store users' passwords and other sensitive data? Some developers are tempted to encrypt passwords, so they can be decrypted for comparison with the password submitted in a login form. Because encryption is designed to be reversed, it is not secure should someone get access to the means for decrypting the data. The Identity framework secures user passwords by *hashing* them. Hashing is a one-way process that should always produce the same output for a given input. For a hashing algorithm to be considered cryptographically secure, it should be infeasible to reverse a hash through brute-force computation, based on the resources (equipment and time) required to do the job. The reality is that processing reduces in cost and increases in power all the time, which has resulted in older hashing algorithms, such as MD5, to become *broken*.

ASP.NET Core Identity uses the PBKDF2 hashing algorithm (which is one of the options recommended by the OWASP). The number of times the actual hashing function is iterated (the *work factor*) determines the difficulty of breaking PBKDF2 hashes. The recommended number of iterations increases all the time as hardware gets faster. In 2010, the number of iterations hardcoded into ASP.NET was 1,000. Currently, the default number of iterations is 10,000, which was the recommended minimum in 2017. The best advice is to increase this number to the largest that the authenticating server's resources will tolerate. The OWASP currently recommends a minimum of 310,000 iterations to conform to the US Federal Information Processing Standards (FIPS). The more iterations that the hashing function is called, the more resources it takes to generate the resulting value. There is a trade-off to be had between generating hashes that are as secure as possible and overloading your web